The Course Master Method

Everything shown or used in class — in the order it comes up

Every document shown or used in class, in the order it comes up. The right-hand column tells you when you need each one; the sidebar bookmarks and the page numbers below both jump straight to any of them.

> **This is not the Course Guide.** The Guide is the course itself — you read it end to end, a few sections a week, and it comes to you as its own file. This is the supporting material we put on screen.

Only the three Guide appendices we actually open in class are reproduced here (Bloom's verbs, the question bank, the craft notes).

Anything you *write in* is in the companion file, **Course_Master_FILL_IN.docx**.

Page	Document	When you need it
3	Participant Syllabus	Start here
7	Session 1 — pre-class	Week 1 — read before class
8	Method on a Page	Week 1 — in class
9	Worked example — Intro-ML: Decisions	Week 1 — in class
10	Worked example — Intro-ML: Objectives	Week 1 — in class
11	Worked example — Intro-ML: MDD (Week 1)	Week 1 — in class
15	Worked example — Coffee: Decisions	Week 1 — in class
16	Worked example — Coffee: Objectives	Week 1 — in class
17	Worked example — Coffee: MDD	Week 1 — in class
20	Worked example — Intro-ML: one segment, full depth	Week 1 — in class
22	Worked example — Coffee: one segment, full depth	Week 1 — in class
25	Guide Appendix A — Bloom's Taxonomy	Week 1 — in class
28	Bloom Verbs Card	Week 1 — in class
29	Directing AI Card	Week 1 onward — your own work
35	Session 2 — pre-class	Week 2 — read before class
36	Seven Practices Card	Week 2 — in class
37	Worked example — Coffee: Seven-Practices Audit	Week 2 — in class
38	Guide Appendix B — Question Bank	Week 2 — in class

Page	Document	When you need it
40	Guide Appendix D — Craft Notes	Weeks 2 and 4 — in class
46	Worked example — Coffee: Decompose	Week 2 — in class
51	Contact-Hour Companion	Weeks 2-3 — sizing your module
57	Session 3 — pre-class	Week 3 — read before class
58	Quality Checklists Card	Week 3 onward — check your work
60	Session 4 — pre-class	Week 4 — read before class
61	Reviewing AI Output — Red Flags	Week 4 — in class
62	Worked example — bounding the answer space	Week 4 — in class
67	Session 5 — pre-class	Week 5 — read before class
68	Recording your teach-back	Week 5 — how the course ends
70	Teaching It Well Card	Week 5 — in class
71	Rubrics 5.1 / 5.2 / 5.3	All weeks — check your work before you hand in
74	Self-Assessment	all weeks
76	Glossary	all weeks

Total: 77 pages.

The Course Master Method — Pilot Syllabus

Dates: October 5 to November 23, 2026 — five sessions, then two weeks to build and record, then feedback **Instructor:** Dr. Lawrence Gray **Class Meetings:** five live sessions on Google Meet at 11 AM MT — Mondays October 5 and 12, Tuesdays October 20, 27 and November 3 (the last two run about an hour). **The module build and the recorded teach-back are due together on Tuesday, November 17**; instructor feedback follows the week of November 23. **Location:** Course materials on GitHub — https://github.com/lawrencegray-brt/how_to_build_a_course
(no account needed; click the green **Code** button, then **Download ZIP**) **Office Hours:** By appointment

---

PROGRAM DESCRIPTION

This five-week pilot teaches subject-matter experts a repeatable method for building high-quality courses and training — the same method behind the program's flagship courses. It is hands-on and production-focused: you bring a real subject you want to teach, and you leave with a real course-in-progress, not notes about course design.

Through instructor-led sessions and guided production between meetings, you will turn a subject into measurable learning objectives, a Master Design Document, and one fully-built module — applying values-first design, Bloom's taxonomy, research-based teaching practices, and GenAI production under guardrails. No prior course-design experience is required.

The pilot mirrors the method it teaches: each module is delivered in the same three-package structure (instructor materials, student materials, a portable teaching guide) you are learning to build, so you experience the method while learning it.

COURSE-LEVEL LEARNING OBJECTIVES

Upon successful completion, you will be able to:

1. Write course- and module-level learning objectives using Bloom's measurable verbs.
2. Audit a course design against the seven research-based teaching practices to find gaps.
3. Produce a complete Master Design Document for a course.
4. Build one fully-realized module across all three packages (instructor, student, teaching guide).
5. Direct GenAI to draft materials and evaluate its output against guardrails for correctness and tone.
6. Embed your organization's values into course activities and assessments.

7. Apply accessibility (UDL) principles so your course works for diverse learners.
8. Plan how to evaluate a delivered course and improve it for the next run.

READINGS AND COURSE MATERIALS

All materials are provided in your Participant Packet:

- **How to Build a Course** — the method guide (your reference all five weeks; Bloom verbs and the practice question bank are in its appendices).
- **Blank templates** — Decisions document, Master Design Document, Seven-Practices audit, and the medium-neutral Module Skeleton.
- **The Contact-Hour companion** — for sizing a module to its time.
- **Two worked examples** — Intro-to-ML (a code course) and Home Coffee Brewing (a non-code course) — copy from whichever fits your subject.

Access to a GenAI tool (e.g., ChatGPT or Claude) is expected for the production work. An **agent-capable** tool — VS Code with Copilot in agent mode, or Claude Code — is optional; it unlocks the steps the Guide and the Directing AI card tag **RUN**, where the AI reads your course folder and does the work itself.

ASSESSMENT

This pilot is assessed on a **completed / not-completed basis — there are no letter grades.** You will receive constructive feedback on every deliverable. Success means producing the three deliverables to the standard of the rubrics (shared in Week 1) and a module you could genuinely hand to someone else to teach.

The three deliverables:

- **A — Learning Objectives** (Weeks 1–2): course objectives first (Week 1), then the module objectives that ladder up to them, written once you decompose into modules (Week 2).
- **B — Master Design Document** (Week 2): every module mapped — objectives, topics, materials, activities, values.
- **C — One complete module, built and taught** (Weeks 3–5): structured (Wk 3), produced with GenAI (Wk 4), and taught as a **10-minute recorded teach-back** of one segment, submitted with the module folder two weeks after Session 5 and returned with instructor feedback on the module rubric. The minimum that counts is one segment, complete.

ASSIGNMENTS

- **Pre-work** — short reading/preparation before each session so live time is spent producing, not lecturing. Includes coming to Session 1 with the subject you want to build a course on.

- **Live sessions** — five required 2-hour Zoom meetings of instruction, guided building, and feedback. Attendance expected.
- **Between-session production** — the real work: drafting your objectives, MDD, and module. In Week 4 you also run a **Review Council** review of your module and revise, keeping a short AI-direction + verification log.
- **Teach-back** — you record a 10-minute segment of your module, taught from your own teaching guide, watch it back once and write a half-page retro, and submit both with your module folder. Feedback is from the instructor, on Rubric 5.3 — met / not yet, no scores. Peer feedback is optional and was not used in the first cohort.

WEEKLY SCHEDULE

Week	Pre-work	Live session (2 hr)	Deliverable / by next week
1 — Decide & Write Objectives	Skim the guide; **bring your subject**	Decisions before production; Bloom objectives; fix-the-weak-objective pair work	**Deliverable A (part 1)** — finish Decisions doc + course objectives
2 — Master Design Document	Read the seven practices; audit your objectives	Feedback on objectives; build your MDD	**Deliverable B** — write module objectives as you decompose, finish the MDD, choose your one module
3 — Structure the Module	MDD final; module chosen	Three-package structure; scaffolding; accessibility (UDL); sketch your module skeleton	Flesh out the skeleton — structure, not content yet

Week	Pre-work	Live session (2 hr)	Deliverable / by next week
4 — Produce With GenAI	Skeleton laid out; Skeleton Map	The production loop — complete first, everything derived; when the AI doesn't know your subject; the loop with an agent (the folder as context, modes, permissions, sub-agents, skill files); the four checks, by name	Build the module to the floor (one segment at minimum); keep the AI-direction + verification log; run the Review Council and the accessibility pass if you get there
5 — Evaluate & Improve, and the Recorded Teach-Back	Keep building; read §9 and *Teaching It Well*	How the course ends; Rubric 5.3 as what the instructor looks for; evaluate & improve with your recording as the first run; Teaching It Well; the close	**Deliverable C** — the module (floor) and a 10-minute recorded teach-back, due together two weeks later; feedback the week after

Pre-Class — Session 1 (Decide & Write Objectives)

~20 minutes. The goal is to arrive ready to work, not to learn it all in advance.

Do this before Session 1

1. **Decide your subject.** Pick one thing you know well and want to teach. Bring it — it's your running project for all five weeks.
2. **Skim the guide, Sections 0–2** (`How_To_Build_A_Course_GUIDE`): the big picture, "decisions before production," and writing objectives. Don't study — just get the shape.
3. **Glance at one worked example's Decisions doc** — `2_Worked_Examples/IntroML/01_Decisions.md` or `2_Worked_Examples/Coffee/01_Decisions.md`. Notice it's one page.

Come ready to answer

- Who is your course *for*, and what should they be able to *do* at the end?

That's it. We build your Decisions doc and objectives live.

The Method on a Page

*A quick-reference to keep beside you — shorthand for once you know the method. New here? Read **Start Here** and the Guide first; this page won't make sense cold.*

The throughline: > Decisions (values first) → Bloom objectives → seven practices → Master Design Document → the three-package module → produce with GenAI under guardrails.

The one rule: Decide before you produce.

The steps

(Step numbers are the conceptual order; the guide section is noted in parentheses.)

1. **Decisions** (§1) — one page: audience, objective, depth, scope, tone, tools.
2. **Objectives** (§2) — what the student can *do*; Bloom verbs; course + module (laddered).
3. **Seven practices** (§3) — audit the design; each answer becomes a feature.
4. **Decompose** (§3.5) — break into right-sized modules that fit your contact hours.
5. **MDD** (§4) — course-header block + one table per module (objectives, topics, materials, activities).
6. **Three packages** (§5) — instructor materials · student materials · teaching guide.
7. **Produce with GenAI** (§6) — persona → solution-first → degrade to practice → ground → verify. You decide; it drafts; you check everything.
8. **Set depth** (§7) — only as deep as the goal needs. The **Review Council** (§6b) reviews each module.

Across it all: design for all learners — accessibility / UDL (§8). **After you teach:** evaluate & improve — close the loop (§9).

The deliverables (what you'll have at the end)

- **A** — Objectives · **B** — Master Design Document · **C** — one module, built & taught.

(In the pilot course these are Deliverables A, B and C. Building on your own, they're simply the three things you finish with.)

The tests

- **Measurable:** can you observe the student doing it?
- **Portable:** could someone else teach this from your materials?

01 — Decisions Document (filled example)

This is the lean, one-page Decisions document for Intro-to-ML. It's the foundation every later choice — and every GenAI prompt — inherits.

- **Audience:** Domain experts (not software engineers); prior stats, basic Python, high-school math. Not expected to know calculus/gradients.
- **Objective:** Production-ready code, not theory. Students *use* gradient descent; they don't derive it.
- **Depth rule:** Only as deep as an MVP project requires. Math is surface-level/visual only.
- **Datasets:** Best canonical dataset per problem type (Week 1 = California Housing). Public datasets only — no proprietary data.
- **Algorithms:** Linear Regression (W1) … XGBoost in; SVM/KNN out.
- **Frameworks:** sklearn (W1–5), Keras 3.x (W6–8).
- **Tone:** Welcoming, whole-person. Hints remind of conceptual purpose; never give the answer.
- **Scaffolding:** Live = complete code → Pair = fill-in-blank (~70% provided W1) → Homework = scaffolded TODOs.
- **Time:** 30 min pre-class + 3 hr live + 30 min post-class.
- **Values:** *Quality* — every practice file is derived from a solution that runs, so students never debug my mistakes. *Humanity* — hints remind, never hand over; nobody is made to feel behind. *Integrity* — we verify AI output in class, because plausible is not correct.

02 — Learning Objectives (filled example)

Bloom-verbed, student-focused, measurable. Module objectives ladder up to the course objectives.

Course-level objectives

By the end of the course, students will be able to:

1. **(Apply)** Implement end-to-end ML pipelines from data loading to evaluation. — *Evidence:* a working notebook that runs load → preprocess → train → evaluate on a fresh dataset.
2. **(Evaluate)** Select appropriate algorithms based on problem type, data, and constraints. — *Evidence:* given a new problem and dataset, justify an algorithm choice against the constraints.
3. **(Create)** Write production-quality code using sklearn Pipelines and Keras training loops. — *Evidence:* submit a clean sklearn Pipeline / Keras loop that runs top to bottom and reads as production code.
4. **(Analyze)** Diagnose common ML failures: overfitting, data leakage, class imbalance. — *Evidence:* from a model's diagnostics, identify the failure and point to the evidence for it.
5. **(Apply)** Communicate model behavior and limitations to non-technical stakeholders. — *Evidence:* produce a plain-language explanation of a model's behavior and limits for a non-technical audience.

Week 1 module objectives (ladder up to CO1, CO4, CO5)

By the end of Week 1, students will be able to:

1. **(Apply)** Execute a complete ML pipeline: load → preprocess → train → evaluate → interpret. *(→ CO1)*
2. **(Apply)** Implement linear regression in sklearn with a proper train/test split. *(→ CO1)*
3. **(Interpret/Understand)** Interpret regression coefficients in business terms ("holding all else constant"). *(→ CO5)*
4. **(Analyze)** Identify residual-plot patterns for diagnostics (random scatter, funnel, curve). *(→ CO4)*
5. **(Evaluate)** Judge model fit using MSE/RMSE/R^2 and communicate results to stakeholders. *(→ CO5)*

Note: objective 3 leans on "interpret/understand" — acceptable here because stakeholder explanation is the measurable outcome (they produce the explanation).

03 — Sample Master Design Document (Georgetown-style)

Filled example using the MDD template. Shows the course-header block + one module table. Conventions: ***→CO#*** *= objective laddering;* **`<Created>`** *= artifact to build; curated media lives in the pre-class study guide.*

Course-Header Block

Field	Content
Course Name	Introduction to ML Programming
Course Description	For domain experts building proof-of-concept ML apps. Production-ready code over theory; intuitive, visual understanding of algorithms sufficient to make decisions and explain them to stakeholders.
Course-Level Objectives	CO1 (Apply) Implement end-to-end ML pipelines. / CO2 (Evaluate) Select appropriate algorithms. / CO3 (Create) Write production-quality sklearn/Keras code. / CO4 (Analyze) Diagnose failures (overfitting, leakage, imbalance). / CO5 (Apply) Communicate models to stakeholders.
Organization values (mapped)	*Quality:* every notebook verified runnable. *Humanity:* welcoming tone, whole-person hints. *Integrity:* honest about model limits/caveats. *Innovation:* GenAI-assisted production.
Teaching Principles (applied)	*Activate Prior Learning:* tie ML to their domain projects. *Organize Knowledge:* same pipeline every week. *Motivate:* canonical real datasets. *Develop Mastery:* gradual-release scaffolding. *Practice & Feedback:* pair work + post-class with solutions. *Climate:* welcoming, debugging-is-normal. *Self-Directed:* cheat sheets, optional deep dives.

Field	Content
Assessment / How Success Is Measured	No grades. Pair-programming + post-class exercises (solutions provided); per-week self-assessment checklists; a capstone project is the ultimate measure.
Materials & Media	Curated videos/readings → each week's pre-class study guide. Frameworks: sklearn (W1–5), Keras 3.x (W6–8).

Module Table

Week 1 — The ML Pipeline & Linear Regression (1 week, 4 hrs)

Module Objectives (→CO#)	Topics (sequential)	Materials & Media	Activities (what students do)
1. (Apply) Execute a full pipeline: load → preprocess → train → evaluate → interpret *(→CO1)*2. (Apply) Implement linear regression with a proper train/test split *(→CO1)*3. (Apply) Interpret coefficients in business terms ("holding all else constant") *(→CO5)*4. (Analyze) Identify residual-plot patterns (scatter, funnel, curve) *(→CO4)*5. (Evaluate) Judge fit with MSE/RMSE/R^2 and communicate it *(→CO5)*	• The ML pipeline (6 steps; split-before-preprocess) • Linear regression intuition (best line) • Loss (MSE) • Residual plots • Coefficient interpretation	Curated media → **Week 1 pre-class study guide** (StatQuest videos; sklearn docs).To build: • `<Created>` live-session notebook (California Housing) • `<Created>` pair-programming notebook (fill-in-blank) • `<Created>` post-class notebook (Diabetes) • `<Created>` glossary, sklearn cheat sheet	• **Pre-class study + warm-up** — watch curated videos; attempt warm-up problems. • **Live follow-along demo** — run the complete California Housing pipeline; read residuals; interpret coefficients. • **Pair programming: "Regression Relay"** — implement the pipeline on a housing variant (fill-in-the-blank, ~70% provided). • **Post-class exercise** — linear regression on Diabetes; residual plot + R^2; bonus: polynomial features. • **Self-assessment** — Week 1 "you should be able to…" checklist.

Additional information: Live-session *minute-by-minute timing* is NOT in the MDD — it lives in the Teaching Guide (see `Week1_folder_skeleton/Teaching_Guide/Day1/Segment1_FULLY_SHOWN`). The MDD names *what* the activities are; the teaching guide sequences *how/when* to run them.

Weeks 2–8

…(same header block carries; one module table per week — omitted in this surface-level example)…

Home Coffee Brewing — Design Decisions (Lean)

Built following Guide §1. One page; the choices everything downstream inherits.

- **Audience:** Adults who drink coffee and want it better at home. No barista/professional background. They own or will buy a basic pour-over kit. Possibly intimidated by "coffee snob" culture.
- **Objective:** Production — by the end they can **brew a good, repeatable pour-over and adjust it to their taste.** Not the chemistry of extraction.
- **Depth:** Only as deep as needed to brew and fix a cup. Extraction is taught as intuition (grind/temp/time → strength & taste), never as chemistry. Stop where going deeper wouldn't improve their cup.
- **Scope — in:** bean freshness & selection, grind size, coffee-to-water ratio, water temp, the pour technique, tasting, and adjusting. **Out:** espresso, milk drinks, latte art, roasting, commercial gear.
- **Values:** *Humanity* — low-shame throughout; a bad cup is data, not failure. *Quality* — there is a reference brew on the bench, so "good" is something you can taste, not an opinion. *Curiosity* — every session ends with one thing to go try and change.
- **Tone:** Welcoming, encouraging, low-shame. Coffee is intimidating to beginners; normalize mistakes and "I'm still dialing it in." Guidance reminds students of *why* a step matters; never condescends.
- **Tools:** Pour-over dripper (V60/Melitta), paper filters, kettle (gooseneck ideal), kitchen scale, burr grinder, timer/phone.

Home Coffee Brewing — Learning Objectives

Built following Guide §2 + Appendix A. Bloom-tagged, student-focused, measurable. Module objectives ladder up to course objectives.

Course-level objectives

By the end of the course, students will be able to:

1. **(Apply)** Brew a pour-over coffee using a repeatable recipe (ratio, grind, time). — *Evidence:* in session, brew a cup that hits your target ratio and time, judged against the reference brew.
2. **(Analyze)** Diagnose a flawed cup (sour, bitter, or weak) from taste and identify the likely cause. — *Evidence:* taste a flawed cup and correctly name the likely cause.
3. **(Evaluate)** Adjust grind, ratio, or technique to improve the next cup toward their own taste. — *Evidence:* given a cup that's off, change one variable and brew a better next cup.
4. **(Understand)** Explain how grind size, water temperature, and ratio affect extraction — intuitively. — *Evidence:* in plain words, explain how grind / temp / ratio push a cup's strength and taste (no chemistry).
5. **(Apply)** Select fresh beans and store them properly. — *Evidence:* pick a fresh bag (roast date, whole bean) and say how you'll store it.

Module 1 objectives — "The Setup & Your First Brew" (→ CO1, CO5)

By the end of Module 1, students will be able to:

1. **(Apply)** Assemble the pour-over setup and measure a coffee-to-water ratio by weight. *(→CO1)*
2. **(Apply)** Grind beans to a medium pour-over grind and rinse the filter. *(→CO1)*
3. **(Apply)** Execute a guided pour-over from bloom to finish, on time. *(→CO1)*
4. **(Apply)** Identify a fresh bean (roast date, whole bean) and store it correctly. *(→CO5)*

*Note: CO4 is an **(Understand)**-level objective on purpose — the audience needs the intuition, not mastery of extraction science. Aim high where the content supports it; don't force a higher verb than the goal needs.*

Home Coffee Brewing — Master Design Document

Built following Guide §4 and MDD_TEMPLATE.md. Course-header block + one module table per module. Conventions: →CO# = objective ladder; `<Created>` = artifact still to build; curated media → the pre-class study guide.

Course-Header Block

Field	Content
Course Name	Introduction to Home Coffee Brewing (Pour-Over)
Course Description	A hands-on beginner course for making café-quality pour-over coffee at home and adjusting it to your taste. Practical and tasting-driven, not chemistry.
Course-Level Objectives	CO1 (Apply) Brew a repeatable pour-over. / CO2 (Analyze) Diagnose a flawed cup. / CO3 (Evaluate) Adjust toward your taste. / CO4 (Understand) Explain grind/temp/ratio → extraction. / CO5 (Apply) Select & store fresh beans.
Values (mapped)	*Quality:* every recipe tested to a reference cup. *Humanity:* low-shame, beginner-friendly, no snobbery. *Integrity:* honest about what's preference vs. what's a mistake.
Teaching Principles (applied)	See `03_Seven_Practices_Audit.md` — one model (grind+water+ratio→extraction→taste); reference-brew benchmark; in-session tasting feedback; brew-log for self-direction.
Assessment / How Success Is Measured	No grades. Success = a brewed cup that hits the reference in session; a completed week-long brew log; passing the tasting self-assessment.
Materials & Media	Curated videos/readings → each module's pre-class study guide. Tools: dripper, filters, kettle, scale, burr grinder, timer.

Module Tables

Module 1 — The Setup & Your First Brew (1 session, ~90 min)

Module Objectives (→CO#)	Topics (sequential)	Materials & Media	Activities (what students do)
1. (Apply) Measure a ratio by weight *(→CO1)*2. (Apply) Grind medium; rinse filter *(→CO1)*3. (Apply) Execute a guided pour-over on time *(→CO1)*4. (Apply) Spot a fresh bean; store it *(→CO5)*	• Gear & setup • Bean freshness • Grind size • Ratio by weight • The pour (bloom → finish)	Curated media → **Module 1 pre-class study guide** (grind-size video; freshness reading).To build: • `<Created>` brew-along guide • `<Created>` recipe-card cheat sheet • `<Created>` reference-brew card (instructor) • `<Created>` glossary	• **Pre-class warm-up** — inspect your beans (roast date? whole?). • **Brew-along demo** — follow the instructor through one full pour-over. • **Paired brew (fill-in)** — complete a blank recipe card from the demo, then brew it with a partner. • **Guided practice** — brew two cups changing ONE variable; compare. • **Self-assessment** — tasting checklist (sour/bitter/balanced). • **Post-class** — brew daily for a week; keep the brew log.

Module 2 — Tasting & Diagnosing (1 session)

…(same header carries; table omitted in this sample — surface-level)…

Module 3 — Dial It In (1 session)

…(omitted — surface-level)…

Teaching Guide — Week 1, Segment 1: Welcome & Roadmap (FULLY SHOWN)

> This is the **one artifact shown at full depth** in the worked example. Everything else in the folder is a stub. Compare this to a stub file to see the difference between "sketched" and "done."

Time: 0:00–0:20 (20 minutes) **Segment goal:** Set the tone, frame what the course is/isn't, and state today's objectives so students know exactly what success looks like.

Segment objectives

By the end of this segment, students will: - Know the course is about **production-ready code**, not math derivation. - Be able to state the five things they'll be able to *do* by end of Week 1. - Feel welcomed (whole-person tone set from minute one).

Materials

- Slide 1 (course roadmap) on the main monitor; this guide on the second monitor.
- Whiteboard for today's objectives.

Verbatim script

(Say it in your own words — this is what to convey, not a script to read robotically.)

> "Welcome to Introduction to Machine Learning Programming. Over the next 8 weeks we go from a simple straight line through data — today — all the way to neural networks.
>
> Let me be clear about what this course **is** and **isn't**. It **is** about building production-ready ML code — you're domain experts building real proof-of-concepts. It is **not** a math course. You will *not* derive gradient descent. You **will** understand what it does well enough to use it and explain it.
>
> By the end of *today*, you'll be able to: 1. Run a complete ML pipeline — load, split, train, evaluate. 2. Train a linear regression model in sklearn. 3. Read residual plots to judge whether the model fits. 4. Explain a coefficient to your boss in plain English.
>
> Quick show of hands — who's used sklearn before? Who's done any ML at all? Perfect, good mix. Let's get started."

Teaching tips

- **Energy sets the tone** — your enthusiasm in the first two minutes carries the room.
- Use names if you can (name tags / Zoom names) — whole-person care starts here.
- Point at the written objectives; refer back to them at the wrap-up.

Anticipated student questions (with answers)

- **"How much math do I really need?"** → "Intuition, not derivation. We focus on patterns and debugging."
- **"Why sklearn and not PyTorch?"** → "sklearn first — simplest path to real pipelines. PyTorch/Keras come in Week 6."

Timing contingencies

- **Running long?** Cut the show-of-hands; just state the objectives and move on.
- **Running short?** Take one extra question or briefly preview the 8-week arc.

Transition to Segment 2

> "You now know where we're headed today. Every ML project — yours included — follows the same pipeline. Let me draw it…" → go to Segment 2: The ML Pipeline.

Teaching Guide — Module 1, Segment 1: Welcome & Why Home Coffee (FULLY SHOWN)

> The one artifact shown at full depth in this sample. Everything else in the folder is a stub. Compare to a stub to see "sketched" vs "done."
>
> This is the **gold tier** at the full anatomy of `1_Guide/templates/Teaching_Guide_Segment_TEMPLATE.md`: timed sub-blocks, inline stage cues, teaching tips, engagement questions, anticipated questions, a segment checklist with a hard timing checkpoint, instructor notes, and a verbatim transition. The floor tier for this same segment would be one agenda row plus three talking points.

Time: 0:00–0:15 (15 min) of a 90-min session. **Segment goal:** Lower the intimidation, give the one mental model the whole course hangs on, and state what they'll be able to do today.

Segment objectives

- Students feel it's OK to be a beginner (low-shame climate).
- Students can state the core model: **grind + water + ratio → extraction → taste.**
- Students know today's goal: brew one good cup, on purpose.

Materials

- The instructor's **reference brew** already made, kept warm, ready to taste (`Instructor_Materials/reference_brew/perfected_recipe_card.md`).
- Small tasting cups, one per student.
- Whiteboard for the one model.

Verbatim script

[0:00–0:04] Open on the shared failure

`[STAND at the front, no slides yet]`

> "Welcome. Quick honesty: almost everyone has made a bad cup of coffee at home — too sour, too bitter, just blah. That's not because you're bad at it. It's because nobody showed you the few levers that matter. Today I'll show you the levers."

`[WAIT for responses — expect nods, maybe a laugh]`

Teaching tip: say this one standing still and unhurried. The point of the first thirty seconds is that nobody in the room is about to be judged. If you rush it,

it reads as a formality and the anti-shame norm doesn't land.

Engagement question: *"Show of hands — who's made a cup at home this week that disappointed them?"* Nearly every hand goes up, and that's the point: they see they're not the only one. Don't make anyone explain it.

[0:04–0:08] The one model

`[WRITE on board, large, and leave it up all session]`

> "Here's the whole course on one line." **"Grind + water + ratio → extraction → taste."** "Everything we do — today and the next two sessions — is just adjusting one of those levers. That's it.
>
> By the end of *today*, you'll have brewed one good cup of pour-over, on purpose, with a recipe you can repeat tomorrow."

Teaching tip: write it, don't slide it. Watching it appear makes it feel small and learnable; a pre-made slide makes it feel like curriculum. Leave it on the board — you will point at it a dozen times today, and every time you do you're reinforcing that there are only three levers, not thirty.

Engagement question: *"Which of those three do you think you've been getting wrong?"* Take two or three answers, don't correct any of them. You're surfacing their priors so they have somewhere to attach today's content — and you'll often hear the exact misconception the module is built to fix.

[0:08–0:15] Taste the target

`[HAND OUT small pours of the reference brew]`

> "First, taste this. This is the target. Notice it's not bitter, not sour — it's balanced and sweet. That's what we're aiming your cup at."

`[PAUSE — let them actually taste before saying anything else]`

Teaching tip: silence here. The instinct is to narrate while they sip; don't. They need one unhurried mouthful of "good" to calibrate against, and your voice competes with it. Then take reactions.

Engagement question: *"How is that different from what you usually make?"* The answers become your diagnostic vocabulary for Module 2 — when someone says "mine's more bitter," you can call back to it two sessions later.

Anticipated questions (with answers)

- **"Do I need expensive gear?"** → "A $30 dripper, a scale, and a burr grinder get you 90% there. We'll talk gear in a minute."

- **"Is pre-ground coffee OK?"** → "It works, but freshness fades fast once ground. We'll cover why — it's the cheapest upgrade you can make."
- **"What if I don't taste the difference?"** → "That's normal and it's trainable — Module 2 is entirely about building that palate. Today just notice whether it's pleasant."

Segment checklist

- ☐ Reference brew made and still warm *before* students arrive.
- ☐ Tasting cups counted out, one per student.
- ☐ The one model written on the board, large, and left there.
- ☐ Anti-shame norm stated out loud, not just implied.
- ☐ Every student has actually tasted the reference cup.
- ☐ **Hard checkpoint: you are at 0:15.** If you're past 0:18, cut the gear discussion in Segment 2 to compensate — do not borrow time from the brewing practice.

Instructor notes

- **Running long?** Drop the bad-coffee story and go straight to the model and the tasting. The tasting is the one thing in this segment that cannot be cut — it's the benchmark the entire module compares against.
- **Running short?** Take one more round of tasting reactions, or ask what they currently use at home.
- **Energy:** this is the warmest segment of the day. Set the tone here and the room stays relaxed through the hands-on work, where people are most likely to feel clumsy.
- **Common pitfall:** instructors skip their own tasting cup. Taste it with them — it signals that you're calibrating too, not grading them.

Transition to Segment 2

> "You've tasted the target. Now let's build the setup that gets you there — starting with the beans." → Segment 2: Gear & Beans.

From **How to Build a Course** — *the full guide has the rest.*

Appendix A — Bloom's Taxonomy (full reference)

Six levels, low to high. Each level's definition and a full list of measurable action verbs. Pick the verb that matches what you want students to actually *do. (Anderson & Krathwohl, 2001 — revised Bloom's.)*

> **Note:** some verbs appear at more than one level (e.g., *choose, compare, select*). That's expected — a verb's level is set by the **cognitive task**, not the word alone. "Compare two options" can be Understand, Analyze, or Evaluate depending on what the comparison demands. Use the level that matches the thinking you want.

Level	What it asks of the student	Measurable action verbs
1. Remember	recall facts, terms, and basic concepts	choose, define, find, identify, label, list, locate, match, name, omit, recall, recognize, relate, select, show, spell, state, tell, what, when, where, which, who, why
2. Understand	explain ideas; organize, compare, interpret	classify, compare, contrast, demonstrate, explain, extend, illustrate, infer, interpret, outline, paraphrase, rephrase, show, summarize, translate
3. Apply	use knowledge in new situations; solve problems	apply, build, choose, construct, develop, experiment with, identify, interview, make use of, model, organize, plan, select, solve, utilize

Level	What it asks of the student	Measurable action verbs
4. Analyze	break into parts; find evidence, motives, relationships	analyze, assume, categorize, classify, compare, conclude, contrast, discover, dissect, distinguish, divide, examine, function, inspect, list, motive, simplify, survey, test for, theme
5. Evaluate	judge against criteria; defend an opinion or decision	appraise, argue, assess, award, choose, conclude, criticize, decide, deduct, defend, determine, disprove, estimate, evaluate, importance, influence, interpret, judge, justify, mark, measure, prioritize, prove, rate, recommend, rule on, select, support, value
6. Create	combine elements into a new pattern; propose alternatives	adapt, build, change, combine, compile, compose, construct, create, delete, design, develop, discuss, elaborate, estimate, formulate, imagine, improve, invent, make up, maximize, minimize, modify, originate, plan, predict, propose, solve, suppose, test

Words to avoid (not measurable)

You can't observe these — replace each with a verb above: *believe, comprehend, conceptualize, experience, feel, grasp, hear, know, learn, listen, memorize, perceive, realize, see, understand* — and phrases like *appreciation for…, awareness of…, familiar with…, knowledge of…, understanding of….*

Not a contradiction: the *level* **Understand** (above) is legitimate — it's the vague *verb* "understand" that isn't measurable. Reach for the Understand-level

verbs (explain, interpret, summarize) instead.

Bloom's Verbs — Quick Card

Pick the verb that matches what students should actually do. Aim high where the content supports it; lower levels are fine when the goal sits there.

Level	Use for	Verbs
Remember	recall facts	define, list, name, recall, label, identify
Understand	explain ideas	summarize, explain, interpret, describe, classify, compare
Apply	use in new situations	apply, build, solve, use, implement, demonstrate
Analyze	break into parts	analyze, distinguish, examine, compare, differentiate
Evaluate	judge against criteria	assess, justify, critique, recommend, defend, rate
Create	make something new	design, develop, compose, propose, formulate, construct

Avoid (not observable): understand, know, learn, appreciate, be familiar with, grasp — *as the objective verb.* Replace with one above.

Test: could you *watch* a student do it? If not, rewrite.

Directing AI — Quick Card

*AI does **seven different jobs** in this method, and only one of them is drafting. Knowing which job you're asking for is most of the skill. Every prompt here is paste-ready; every one has a verification step, because **the AI advises — you decide and verify.** Some jobs also have a second form — **RUN** — where an agent reads your course folder and does the work itself; it is marked below wherever it applies. Every prompt carries the id of its Guide callout — **AI-3.1** is the first callout in §3 — and the Teaching Guide refers to prompts by id.*

The seven jobs — and RUN

Job	What you're asking for	When
TUTOR	explain something *to you*	any time, every section
WIDEN	what am I missing?	after you've drafted, before you commit
CHECK	test what I decided against a standard	after a draft exists
SIMULATE	be my student, so I can see it from their side	after a draft exists
TRANSLATE	change the form, add nothing	once content is final
CARRY	hold my decisions as standing context	from §1 onward, continuously
PRODUCE	draft the artifact	**§6 / Week 4 only**
RUN	do one of the jobs above *with my course folder in view*, and loop until it's done	any week — **it inherits the job's timing:** RUN · CHECK any time; RUN · PRODUCE is still Week 4

Notice PRODUCE is last and alone. AI drafts nothing before Week 4 — that is *decisions before production*, not an oversight. Before then it checks, widens, simulates and tutors.

RUN keeps its job tag — RUN · CHECK, RUN · PRODUCE — because the timing rule attaches to the job, not to RUN. An agent that *checks* is welcome in Week 2; an agent that *drafts* waits for Week 4 like everything else. A chat answers; an agent works — it reads your files instead of what you pasted, runs the work, checks the result and fixes it before you see it. What it never does is decide.

The job AI does NOT get: ESTIMATE. "How long will this take?" is the question it is worst at and most confident about. Use the Contact-Hour companion and your own measured pace. *(Agents are told this in writing — see the instruction file under CARRY.)*

Always available

AI-0.1 · TUTOR > "I'm a subject-matter expert, not an instructional designer. When I hit a term I don't know — gradual release, altitude, scaffolding, UDL — explain it using **my own subject** as the example, and tell me which section it matters in."

AI-1.1 · CARRY — chat: paste once, at the start of every working session > "This is my Decisions doc. Treat it as fixed context for everything I ask from now on — don't make me restate the audience, depth or tone."

If a later draft contradicts your Decisions doc, the context slipped. Re-paste it.

AI-1.1 · CARRY · RUN — nothing to paste. Put `AGENT_INSTRUCTIONS.md` (in `templates/`) at the root of your course folder, renamed `AGENTS.md` — the universal name; Copilot, Codex and most agents read it, Claude Code reads `CLAUDE.md` — and the agent reads Decisions, Objectives and the MDD before every task. Nothing is pasted, so nothing can slip.

Every RUN prompt has the same shape: name the files to read, name the files to write, say when to stop, say what not to touch. *"Change nothing"* is the agentic form of *"do not rewrite them."*

Week 1 — objectives (Guide §1–§2)

AI-1.2 · WIDEN — what's missing from my audience? > "My audience is [X]. Who else realistically ends up in this room, and what would they need that my description doesn't cover?"

AI-2.1 · CHECK — the four traps > "Here are my objectives. For each, say which it falls into — instructor-focused, activity-not-learning, not measurable, inflated verb — or 'none'. **Do not rewrite them.**"

The last sentence is the guardrail. Without it, it writes your objectives and the decision has left the room.

AI-2.2 · WIDEN — what objectives am I missing? > "Here is my Decisions doc and my objectives. For this audience, goal and depth, what would a student in a course like this usually be able to **do** by the end that none of mine names? Give each as a measurable objective at the depth in my Decisions doc — no 'understand' or 'know'. **Don't rewrite mine.**"

*Input, not instruction. You do **not** have to add any of them — the job is to make sure you're excluding things on purpose, not by accident.*

AI-2.3 · SIMULATE — is my objective ambiguous? > "Here is one objective. Write three different things a student might hand in as evidence they met it."

*If what comes back isn't what you had in mind, **the objective is ambiguous** — and you found out before anyone was assessed on it.*

Week 2 — audit, decompose, MDD (Guide §3–§4)

AI-3.1 · WIDEN — stuck on a practice > "Here are my Decisions doc and my course objectives. I'm auditing them against the practice '[anchor question]'. Name the features my course is missing that would answer it — concrete activities, materials or pieces of content, each tied to the objective it serves and fitting the audience and depth in my Decisions doc. Five at most. I'll pick or reject."

*The practice examines **your** objectives and decisions — so they go in, not your topic. A feature you reject is still a decision.*

AI-3.5.1 · WIDEN — what modules am I missing? > "Here are my course objectives and my module list. What modules would a course like this usually have that I don't? **Don't reorganise mine.**"

AI-3.5.2 · SIMULATE — test the sequence > "You're a student who just finished Module 3. Based only on that, what do you expect Module 4 to teach? What would confuse you if it came next?"

If its expectation doesn't match your Module 4, your order has a seam.

AI-4.1 · CHECK — the trace-back test, run cold > "Here are my course objectives and my module list. Name any module that doesn't trace to an objective, and any objective no module serves."

For each flag: scope creep to cut, or an objective you forgot to write?

AI-4.1 · CHECK · RUN — the same test, over your folder > "Read the Objectives doc and the MDD in this folder. Report each →CO# tag that points at no course objective, and each course objective no module serves. List only, with file paths. Change nothing."

*The first RUN of the course, and it's read-only. Whatever it lists is a question, not a verdict. (The whole-course drift check, once modules exist, is **AI-6.7** — Guide §6e.)*

AI-9.3 · TRANSLATE — the syllabus falls out > "From this Decisions doc and MDD, draft the student-facing syllabus. Every fact must come from one of them."

Week 3 — structure and access (Guide §5, §8)

AI-5.1 · WIDEN — make a vague activity buildable > "Here is my Decisions doc. Module objective: [X]. My ramp is: I do = [the demo], we do = [the paired fill-in], you do = [the independent task]. Give me three concrete named activities for the 'we do' rung that fit this audience and depth — each with what's provided and what the student supplies, blanking decisions rather than syntax. **Name them; don't write them.**"

AI-5.3 · SIMULATE — find the missing rung > "Here's my complete version and my practice version. You're a student who has only seen the complete one. Attempt the practice. Where do you not know what to do?"

AI-5.2 · CHECK — portability > "Read this teaching guide as someone who has never taught this subject. List every place you'd have to improvise."

*This is the real bar: could someone **else** teach it from the page alone?*

AI-8.1 · PRODUCE — alt-text for every visual > "Here is my module text and a list of its visuals with what each shows. For each, draft alt-text that conveys what the visual **teaches**, not just what it depicts."

AI-8.3 · CHECK — the one-mode-only audit > "Flag any concept in these materials reachable only one way — only by reading, only by a diagram, only by hearing me say it."

AI-8.4 · WIDEN — barriers I haven't considered > "Here is my module and my Decisions doc. For this audience, where does the material assume one way to perceive it, one way to engage with it, or one way to show what was learned? Name the assumption and where it sits."

AI-8.2 · TRANSLATE — captions and transcripts > "Turn this segment script into captions and this demo into a written transcript. **Add nothing that isn't in the source.**"

AI-8.3 · RUN — all of §8 as one sweep > "Read every file under this module folder. Report each concept reachable only one way (only text, only a diagram, only spoken in the script), with file and location. For each visual, draft alt-text that conveys what it teaches. Turn the segment scripts into captions and the demo into a transcript, adding nothing not in the source. Write all drafts to `accessibility/`; edit no existing file."

The sweep saves you the pasting. It does not save you the reading — every alt-text line still gets read against its image.

Week 4 — produce (Guide §6)

This is PRODUCE's week, and the Guide walks the loop step by step — set the persona, build the complete version, degrade it into practice, bound the answer space, verify every line. **Use §6a, not this card.** The two councils and the comprehensiveness check are in §6b and §6c. **With an agent, §6e has the same loop as seven prompts, AI-6.1 to AI-6.7** — build-degrade-verify (6.1), bounding (6.2), the Design and Review Councils (6.3, 6.4), the deck (6.5), the comprehensiveness check (6.6), the whole-course drift check (6.7) — and the one rule that matters most there: *the agent's report is not your verification.* Keep the log (`AI_Direction_Log_TEMPLATE`) either way.

Week 5 — evaluate (Guide §9)

AI-9.1 · SIMULATE — a pre-mortem, before you have real feedback > "Here is my Decisions doc and my module. You are one of the students it describes, and it didn't work for you. You're not hostile, just honest. What went wrong?"

A prompt for what to watch for — never a finding.

AI-9.2 · TRANSLATE — retro notes into real changes > "Here are my notes from the run. Turn them into specific changes, each naming the MDD row or module file it lands in. **Don't propose anything my notes don't support.**"

AI-9.2 · TRANSLATE · RUN — the same, as a diff against your folder > "Read my retro notes — `retro/run_<date>`, in a `retro/` folder beside the modules — and the whole course folder. Turn each note into a specific change that names the MDD row or module file it lands in. Propose nothing the notes do not support. Write the proposals to `retro/proposed_changes.md` as a diff I can accept or reject line by line. **Apply nothing.**"

The three rules under all of it

You decide and verify. Every prompt above ends with something you check. "The AI wrote it" is never an excuse for an error, a stale fact, or a tone you'd never use.

Plausible is not correct. Two red flags tell you where to dig: **verbosity** (it won't stop explaining) and **certainty** (hedge-free). Where it sounds most sure and over-explains, check hardest. *(Full card: Reviewing AI Output — Red Flags.)*

The report is not the verification. An agent that closes its loop and says green has proved it *runs*, not that it *teaches*. Paste its "could not verify" list

into your log and check every line yourself — and never let one agent be the checker of its own work.

Pre-Class — Session 2 (Master Design Document)

~30 minutes.

Do this before Session 2

1. **Finish your Decisions doc and objectives** (Deliverable A) if you haven't.
2. **Check your objectives — this is the objectives feedback.** There is no feedback round in class, so this step has to happen before you walk in. First check each course objective yourself against **Rubric 5.1** in your packet: a measurable verb · student-focused · the right altitude · assessable. (*Laddered*, the fifth line, waits until you have module objectives.) Then run the three prompts from Session 1, all on the **Directing AI** card by id: **AI-2.1 CHECK** the four traps (*"…or 'none'. Do not rewrite them."*), **AI-2.2 WIDEN** (*"what would a student in a course like this usually be able to do that none of my objectives names? Give each as a measurable objective."*), **AI-2.3 SIMULATE** (*"write three things a student might hand in as evidence they met it"*). Fix what you agree with. **Bring what you disagreed with** — there is a question slot at the end of class, and those are the interesting ones.
3. **Read the guide, Section 3 (Seven Practices)** and **Section 3.5 (Decompose Into Modules)**. Skim **Section 4 (the MDD)**.
4. **Run the Seven-Practices audit on your own course** — use `templates/Seven_Practices_Audit_TEMPLATE.md`. For each practice, write your answer as a concrete feature. Where you have no answer, that's a gap to bring to class. **Do your own pass first;** then, if you like, put an LLM to work on the practices you blanked on — **AI-3.1** on the Directing AI card: your Decisions doc and objectives in, features that serve *those* objectives out — and *widen* ("given these objectives and this audience, what am I still missing?"). Keep only what fits your audience and rewrite it in your voice. *(Unsure how to build a good feature? The guide's **Appendix D** has a craft note for each of the seven.)*
5. **Glance at the filled coffee pair** — `2_Worked_Examples/Coffee/03b_Decompose.md` and `04_MDD.md`. In class you will see both walked step by step. **You build your own during the week after class, not in class** — decomposing a course and writing its design document take real thought, and the finished example is there to work beside.

Come ready to

- Bring your practices audit (gaps included), and the objectives you disagreed with the AI about.

Seven Practices — Quick Card

*Audit your design. For each, answer the question **as a concrete feature you'll build.** No answer = your next design task. (Full question bank: Guide Appendix B.)*

1. **Activate Prior Learning** — What analogy connects what they already know to this?
2. **Organize Knowledge** — What's the relationship among the key concepts? (Can you draw a map?)
3. **Motivate** — What excites you about this? What would excite them?
4. **Develop Mastery** — What model shows the performance level to aim at?
5. **Practice & Feedback** — How do they practice *before* being assessed?
6. **Student Development & Climate** — What traits does a strong practitioner have — and does the course cultivate them?
7. **Self-Directed Learners** — What lets them keep learning after the course ends?

Home Coffee Brewing — Seven-Practices Gap Audit

Built following Guide §3. For each practice: my answer as a concrete course feature. Where I had no answer, it became a design task (noted).

Practice	Answer (a concrete feature)
Activate Prior Learning	Everyone has made *bad* coffee — open by having them recall a disappointing cup; we'll explain why it happened.
Organize Knowledge	One mental model used all course: **grind + water + ratio → extraction → taste.** Every lesson maps to a lever on that model.
Motivate	The hook: "café-quality coffee at home for pennies a cup." Real beans, real tasting in session 1.
Develop Mastery	The instructor's **reference brew** is the benchmark; students taste it, then aim their own cup at it.
Practice & Feedback	Paired brew-along in session; tasting feedback on the spot; a week-long brew log after.
Student Development & Climate	Normalize "still dialing it in"; group tasting builds a low-stakes, shared-learning vibe; no snobbery.
Self-Directed Learners	A recipe-card cheat sheet, a tasting checklist, and pointers to good coffee communities/resources so they keep improving.

Gap found during the audit: I had no good answer for *Develop Mastery* until I added the **instructor reference brew** as the explicit benchmark — that became a required instructor artifact. (This is the audit doing its job.)

From **How to Build a Course** — *the full guide has the rest.*

Appendix B — The Seven Practices: Full Question Bank

Lead with the anchor question (Section 3); reach here when a section feels thin. *(Adapted from the Georgetown SCS Course Design Framework.)*

1. Activate Prior Learning

- What examples/analogies connect what students already know to this content? *(Build two: one for the* ***intuition****, one aimed at the* ***misconception*** *— the place they predictably go wrong, with an everyday image that makes the* wrong *turn feel wrong.)*
- What should students already know before this course?
- Which concepts build on knowledge students should already have?

2. Organize Knowledge

- What is the interrelationship among the key concepts?
- Can you draw a concept map of how they relate and build on one another?
- How do the key concepts build on each other — and on prior knowledge?

3. Motivate

- What excites you about this course? What would excite students?
- What stories or real-life examples help explain this content?
- How could a student apply this to their job? Examples from their profession?
- Why is this content important for them to know?

4. Develop Mastery

- What examples/models reflect the performance level to benchmark against?
- What examples reflect *poor* performance?
- Can the work be broken into smaller chunks across the term?
- Under what real conditions would students do a similar task?
- How does this prepare them for later content?

5. Practice & Provide Feedback

- How do students practice before being assessed?
- How can instructor feedback come *before* the final product?
- What goals would you give them to practice toward?

6. Student Development & Course Climate

- What traits do highly effective practitioners in this field have?
- What professional organizations/forums relate to this content?

- What examples reflect diversity and inclusion?
- What discussion questions are pertinent? How might students connect socially?
- What group activities help them acquire the skills?

7. Self-Directed Learners

- Can you share assignments that highlight strong vs. weak performance?
- What resources let them keep learning after the course?
- What rules-of-thumb can students use to assess their own work?
- What should they reflect on to analyze their own performance?
- Where in the material should they pause and self-evaluate?

From **How to Build a Course** — *the full guide has the rest.*

Appendix D — Building the Seven Features (Craft Notes)

The seven-practices audit (§3) tells you ***what*** *features your course needs; these tell you* how *to build a good version of each — one note per practice: why it's high-leverage, the moves, what AI can and can't do, and a worked example. Pull the one you need when a feature feels thin.*

Analogy Craft — teaching to explain simply

Analogy is how a hard idea becomes a simple one, and it may be the highest-leverage thing you do at the board — so it gets its own practice. A concept worth teaching earns **two** analogies, each doing a different job:

- **Analogy #1 builds the intuition** — it connects the new idea to something the audience already lives in.
- **Analogy #2 targets the misconception** — the specific place learners predictably go wrong — and bends the picture so the *wrong* turn feels wrong.

The skill underneath is **teaching to explain simply**, and it's worth practicing on purpose — and handing to students, because making *them* build the analogy is how you find out whether they actually understand the thing. AI can *generate* candidates (see §6, *Draft candidate analogies*), but a borrowed analogy in someone else's voice falls flat: **you** keep the one or two that ring true, rewrite them into your world and your audience's (a tech for tractors hears it differently than a data scientist does), and check that the mapping actually holds.

> **Worked example — backpropagation as seasoning a chicken.** Training a model is like seasoning a chicken. You add a little salt and pepper, you taste, and the taste tells you *which way you were off* — too bland, too salty — so you adjust and taste again, closing in on right. That's backprop: **the seasoning is the weights** you're tuning, **the tasting is evaluating the loss** (how far off the dish is), and **the adjust-and-taste-again is the gradient step,** nudging each weight the direction the taste says to go. Round after round, the seasoning dials in. *Analogy #2, for the misconception:* people think the model lands on the answer in one shot. It doesn't — nobody seasons perfectly on the first shake. You *iterate*: taste, adjust, taste, adjust. The error isn't a failure; it's the information that tells you which way to move. Notice the move — warm, concrete, *theirs* (everyone has over-salted something), with a precise mapping underneath. That's the whole technique.

Structure Craft — turn a pile of facts into a map

Experts hold a subject as a *connected structure*; novices hold it as a *pile of facts* — and the pile is why they can't retrieve or transfer what they learned, relearning every case from scratch. Organizing the knowledge is what turns "I memorized twenty things" into "I see how this fits," and it does double duty: the same map that helps students is what shows *you* the natural module boundaries (see §3.5). The moves:

- **Links are labeled *relationships*, not just lines.** A concept map is concepts joined by *named* relationships — "requires," "is a kind of," "causes," "trades off against." The label is where the learning lives: *"grind size → controls extraction → drives taste"* teaches more than the three terms sitting side by side.
- **Find the spine first.** Every domain has a backbone — a process, a cycle, a hierarchy, a cause-effect chain, a spectrum. Name the one organizing principle and hang everything off it. A map with no spine is just a tangle.
- **Show the whole before the parts.** The map isn't only your design tool — give students a version of it (a diagram, an advance organizer at the module intro, a "you are here"). When they see the structure first, each new piece has a place to land.
- **Test it by navigating.** A good structure lets a learner answer *"where does this fit?"* and *"what connects to what?"* If a concept has no links, or everything links to everything, the structure is wrong — fix it before you build content on top of it.

AI will happily *draw* a concept map — but it over-connects (everything relates to everything) and rarely picks the *one* spine that matters for your audience. Use it to surface candidate concepts and relationships; **you** choose the spine and prune to the links that carry weight.

> **Worked example — coffee, pile vs. map.** The pile: *grind, water temperature, ratio, brew time, bean freshness* — five variables to memorize. The map: put **extraction** on the spine, and every variable becomes a labeled lever on it — *"grind finer → faster extraction → bitter if you overshoot," "cooler water → slower extraction → sour if you undershoot."* Now a beginner can *reason*: a sour cup means under-extraction, so grind finer or go hotter. The labeled cause-effect map *is* the understanding — and "extraction" is exactly the boundary that makes it one coherent module.

Motivation Craft — make them care before you teach

A learner who doesn't care won't learn — attention is the gate everything else passes through, which makes this the practice people most often *wave at* ("this matters because it's foundational") instead of *building*. The research frame is **ARCS** (Keller): **A**ttention, **R**elevance, **C**onfidence, **S**atisfaction. In practice it's four moves:

- **Make relevance personal, not field-level.** "X is important in today's economy" motivates no one. Connect to a stake *this* learner already has — a problem they hit, a goal they want, a frustration they feel. The test: can they finish *"I want this because ___"* in their own words?
- **Lead with the payoff, not the prerequisites.** People are pulled by the destination and repelled by the runway. Show the useful end result *first* — the working app, the café-quality cup — then say what it takes. Don't make them earn the motivation by slogging through fundamentals.
- **Open on a real tension.** A hook works when learners *feel* the gap between where they are and where they want to be. "Everyone's made a bad cup of coffee" surfaces a pain the course then resolves.
- **Engineer an early win.** Nothing motivates like "I just did that." Design one real, reachable success in the first session (ARCS *Confidence*) — it buys you patience for the harder parts.

AI can draft candidate hooks, but relevance is **audience-specific** — one that lands for a data scientist falls flat for a tech maintaining tractors. You supply the audience truth; AI generates options; you keep the one that rings true for *your* people.

> **Worked example — a weak hook vs. a strong one.** Weak: *"Machine learning is a foundational skill in today's economy."* (Field-relevance, no personal stake, no payoff, no tension — nobody leans in.) Strong: *"By the end of today you'll have a model running that predicts house prices from real data — and you'll see exactly where it's wrong and why. No calculus. Now think of a prediction* you'd *actually want to make at work — we'll build toward that."* Notice the moves stacked: payoff first ("a model running today"), an early win ("by the end of today"), confidence ("no calculus"), and relevance made personal ("a prediction *you'd* want to make").

Mastery Craft — show them what "good" looks like

People can't hit a target they can't see. Novices don't know what *good* looks like, so they can't aim for it or judge how far off they are — showing a complete, expert-quality example at the level to reach is one of the highest-return moves in teaching (the *worked-example effect*: studying a solved example beats grinding one out from scratch). The moves:

- **Show the whole, finished thing — the benchmark.** Not a fragment; the real deliverable, done well, so "good" is concrete. The Worked Examples that ship with this guide *are* this move — a full module at the bar to aim at.
- **Make the standard explicit.** Don't just show it — say *why* it's good (the two or three things that make it work), so learners can judge their own against the same criteria.
- **Show the process, not only the product.** Reveal how an expert got

there — the false starts, the checks, the "I'd redo this." Otherwise learners think experts nail it first try, which is both false and demoralizing.
- **Then fade it.** They reproduce it with support, then without — that's the scaffolding ramp (§5), and the benchmark is where the ramp starts.

AI drafts a plausibly-good example fast, but *good* is domain- and audience-specific; it tends to hand you the generically-competent version that misses what your field actually prizes. You set the bar and verify the example is genuinely exemplary — an off benchmark teaches the wrong target.

> **Worked example — the reference brew.** The coffee course had no model of mastery until the audit forced one: an **instructor reference brew** students taste at the start (*this* is what you're aiming at) and taste their own against. For a code course it's the complete, verified pipeline notebook — the thing that runs, that they build toward.

Practice Craft — let them try before it counts

You don't learn a skill by watching someone do it — you learn by doing it, getting feedback, and doing it again, *before* the stakes are real. It's the step most courses skip: they explain, then assess, with no safe middle. The moves:

- **Practice the objective's actual verb.** If the objective is to *implement* a pipeline, they practice implementing — not answering quiz questions *about* pipelines. Practice that doesn't match the verb builds the wrong skill.
- **Keep it low-stakes.** Practice is for making mistakes cheaply; the moment it's graded, it's assessment, not practice. Protect a space where being wrong costs nothing.
- **Make feedback timely and specific.** "Good job" isn't feedback; *"your split leaks test data into training — here's why that inflates the score"* is. Feedback that arrives after it counts is too late to learn from.
- **Degrade it from the worked version.** The cleanest practice is the benchmark with holes cut in it (fill-in-the-blank, TODOs) — because it's a working thing with parts removed, the practice can't be subtly wrong.

AI generates practice items and even feedback well, but it pitches difficulty unevenly and defaults to generic notes; you set the difficulty ramp and aim the feedback at the specific misconception. *(Fuller build: the scaffolding ramp, §5, and* build-complete-then-degrade*, §6.)*

> **Worked example — the brew-along.** A paired brew-along where the *tasting is the feedback* — immediate, specific, low-stakes — then a week-long brew log to practice alone before anything is assessed.

Climate Craft — make it safe to be a beginner

Fear shuts learning down. A learner afraid of looking stupid won't ask the question, try the risky thing, or admit they're lost — and every one of those is required to learn. Climate is the invisible variable that gates all the others, and it's where you grow the *person* into a practitioner, not just fill them with content. The moves:

- **Normalize struggle and error, out loud.** Say that confusion and mistakes are expected, and model your own — "debugging is normal," "nobody seasons perfectly on the first shake." A visible instructor misstep gives everyone permission.
- **Design for every learner, not the median.** Vary your examples (not all sports or war metaphors), don't assume background that quietly excludes, and offer more than one way in (this is UDL, §8). Ask: whose experience does each example assume?
- **Set the norms explicitly.** How questions work, how feedback is given (specific *and* kind), and the flat rule that *no question is too basic*. Say it in week one; hold to it.
- **Name the practitioner's traits and build reps for them.** Curiosity, skepticism, persistence — decide which a strong practitioner in your field has, then design moments that *exercise* them, not just mention them.

AI can scan your examples for assumed knowledge and suggest inclusive alternatives, but climate is set by *you* — in the room and in the tone of every artifact. It's a design *value* (see §0), not a feature bolted on at the end.

> **Worked example — open on a shared failure.** The coffee course opens by having everyone recall a bad cup: now no one is the only novice, mistakes are the *subject*, and the room is safe to be wrong in. "Debugging is normal" does the same job for a code course.

Self-Direction Craft — set them up to keep going without you

A course ends; learning shouldn't. The real win is a learner who keeps improving after you're gone — anything less builds dependence, not capability, and that's what separates a course from a crutch. The moves:

- **Hand over your tools.** The cheat sheets, checklists, reference examples, and "how I'd debug this" heuristics you used — give them, so learners self-serve instead of coming back to you.
- **Teach the meta-skill, not just the answers.** How to find the answer, judge a source, run the next experiment — the fishing rod, not the fish. A learner who can *figure things out* outlasts one who memorized this year's facts.

- **Point to the next rung.** A curated "where to go deeper" — optional dives, a reading or two, the community — so momentum survives the last session instead of dying at the door.
- **Build in self-assessment.** A way for them to gauge their *own* understanding (the module's self-check) so they can steer without you grading them.

The highest-leverage take-home now is teaching them to **keep learning with AI** — under the same guardrails you use in this course (it proposes; you verify). Model it, and they leave with a tutor that never closes.

> **Worked example — the take-home kit.** Coffee: a recipe card + a "how to dial in *any* coffee" heuristic + a short resource list. Code: the cheat sheet + a "how to debug a model" checklist + pointers to the next topics — plus the habit of asking AI well.

> **Now go build one.**, Open `templates/Decisions_TEMPLATE.md` and fill the first page — audience, objective, depth. That single page is what everything in this guide hangs from, and it takes about twenty minutes. **Build one module end to end before you scale to the rest**: the first one is slow because you're learning the method, and every one after it inherits the decisions you just made.

Decompose Into Modules — WORKSHEET (filled: Home Coffee)

Guide §3.5. Blank form: `1_Guide/templates/Decompose_TEMPLATE.md`*. This sits between the seven-practices audit (*`03_Seven_Practices_Audit.md`*) and the MDD (*`04_MDD.md`*) — its output table becomes the MDD's module rows.*

Container: three 90-minute sessions = 4.5 contact hours. No homework culture assumed; pre-class is optional.

Steps 1–2 — each course objective → evidence → skills → module objectives

Do these two steps for ***every*** *course objective before clustering anything. That ordering is the whole trick: skills from different objectives turn out to belong in the same module, and you can't see that if you decompose one objective at a time.*

Course objective	Evidence (how they prove it)	Skills → module objectives (→CO#)
CO1 — brew a pour-over to a repeatable recipe	In session, brew a cup hitting the target ratio and time, judged against the reference brew	measure a coffee-to-water ratio by weight (→CO1) · grind to the right texture and rinse the filter (→CO1) · execute the pour on time, bloom included (→CO1)
CO2 — diagnose a flawed cup	Given a bad cup, name whether it's sour, bitter or weak and say why	taste for sour / bitter / weak and name it (→CO2) · connect each fault to its cause (→CO2)
CO3 — adjust toward their own taste	Given an off cup, change one variable and brew a better one	change one variable at a time (→CO3) · predict the direction of the change before brewing (→CO3)
CO4 — explain extraction intuitively	In plain words, explain how grind / temp / ratio push extraction	explain extraction as a dial, not chemistry (→CO4)

Course objective	Evidence (how they prove it)	Skills → module objectives (→CO#)
CO5 — buy and store beans well	Choose a bag and say why; store it correctly	read a roast date (→CO5) · store beans so they stay fresh (→CO5)

Step 3 — cluster the whole pool into modules

Fourteen skills, three sessions. Clustering by *what you'd do in one sitting*, not by which objective they came from:

- **Everything needed to make one cup at all** — ratio, grind, rinse, pour, bloom, timing — has to come first; nothing else can be practised until they can brew.
- **Bean selection and storage (→CO5)** is small and doesn't need its own session. It rides along in Module 1, because buying beans is the thing they'll do *that week*, before session 2.
- **Tasting and diagnosing (→CO2) and explaining extraction (→CO4)** belong together: you can't diagnose a cup without the extraction intuition, and the intuition is abstract without a flawed cup in hand.
- **Adjusting (→CO3)** is last because it needs both: you can't dial in a cup until you can brew one *and* taste what's wrong with it.

Note the many-to-many fan-out: CO1 contributes three objectives to Module 1, CO5 rides along in the same module, and CO2/CO4 share Module 2. Five course objectives, three modules — the mapping is not one-to-one, and it shouldn't be.

Step 4 — sequence by prerequisite

```
Module 1 (brew) → Module 2 (taste & diagnose) → Module 3 (dial in)
```

Strictly forced: each module's practice needs the previous module's skill. No reordering is possible here, which is a good sign the clustering is right.

Step 5 — surface each module's features (seven practices)

From the audit (`03_Seven_Practices_Audit.md`), the features each module needs:

Module	Features the audit surfaced
1	Opening "taste your usual cup" warm-up (*Activate Prior Learning*, *Motivate*) · the **instructor reference brew** as the benchmark (*Develop Mastery*) · paired fill-in brew (*Practice & Feedback*) · low-shame framing — a bad cup is data (*Climate*)
2	Deliberately flawed cups to taste (*Develop Mastery*) · the extraction dial analogy (*Organize Knowledge*) · partner diagnosis out loud (*Practice & Feedback*)
3	One-variable-at-a-time experiment (*Practice & Feedback*) · "how to dial in any coffee" heuristic to take home (*Self-Directed Learners*) · brew log (*Self-Directed Learners*)

Step 6 — coarse fit-check

Ballpark only — real minutes come at §6a step 6, once the materials exist. Using the Contact-Hour Companion against a 90-minute container:

#	Rough content	Ballpark	Verdict
1	demo + 2 guided brews + bean talk	~85 min	fits, tight
2	3 flawed cups + diagnosis + analogy	~75 min	fits
3	2 experimental brews + debrief	~80 min	fits

Shared equipment is the real constraint, not content: with one dripper per pair and six pairs, Module 1's two brews are the whole session. If there were only two drippers for the room, this module would not fit and would have to split.

Step 7 — iterate

One change came out of the fit-check: bean selection and storage was originally its own fourth module. At ~25 minutes of content it was far too thin for a 90-minute session, so it merged into Module 1 as a closing segment — which is also better pedagogy, since they buy beans before session 2.

The module list — the output (→ becomes the MDD rows, §4)

#	Module	Module objectives (→CO#)	Evidence it builds toward	Features (seven-practices)	~ Contact time	Fit?
1	The Setup & Your First Brew	measure a ratio by weight (→CO1) · grind & rinse (→CO1) · execute a pour-over on time (→CO1) · spot & store fresh beans (→CO5)	A cup hitting target ratio and time, judged against the reference brew	warm-up tasting · reference brew · paired fill-in · low-shame framing	~85 min	fits
2	Tasting & Diagnosing	diagnose sour / bitter / weak (→CO2) · explain extraction intuitively (→CO4)	Name the fault in a given cup and say why	flawed cups · extraction dial analogy · partner diagnosis	~75 min	fits

#	Module	Module objectives (→CO#)	Evidence it builds toward	Features (seven-practices)	~ Contact time	Fit?
3	Dial It In	adjust grind, ratio or technique toward your taste (→CO3)	Change one variable and brew a better cup	one-variable experiment · take-home heuristic · brew log	~80 min	fits

These three rows become the three module tables in `04_MDD.md`.

Contact-Hour Companion — Sizing a Module to Its Time

*The tool for two moments in the build: the **coarse fit-check** in §3.5 (Decompose Into Modules) — a ballpark size check from a module's objectives and depth, before any materials exist — and the **precise time budget** in §5 (Structure the Module), once the materials are real and you can sum actual minutes. Either way it estimates how long a module's work takes, so you can check it fits your container. Built on the Georgetown SCS Contact Hour Assessment, **re-pointed for short professional workshops** (not 15-week academic courses).*

Read this first: these are not academic courses

The original Georgetown tool sizes a **15-week graded academic course** by its *weekly out-of-class workload* — a 7–9 hr/week target full of discussion-board posts, reflection papers, and HBR readings. **Your courses are different:**

- They're **defined by contact time**, not weekly homework load. The session length *is* the container.
- They're **short and varied** — a 1.5-hour intro, a 3-hour workshop, a 5-week 2-hr/week series.
- They're **hands-on and ungraded** — practice and demos, not graded discussion posts.

So you use the tool's **mechanism** (estimate time, sum, compare to the container) and its **reading-speed benchmarks**, but you **drop the academic furniture** (discussion-post counts, reflection-paper hours, weekly-workload targets). The question is simply: **does what I planned fit the time I actually have?**

The tool works in both directions

A "container" is your course's real schedule, measured in **total contact hours.** You don't have to use any standard format — **design whatever shape fits your goal.** This tool just lets you reason about time both ways:

- **Time-first** — *"I have this much time. What can I realistically cover?"* (Fit the content to the container.)
- **Content-first** — *"I want to teach this. How much time will it take?"* (Let the content size the container.)

Most builds bounce between the two until content and time agree.

Example formats (these are formats I have used — yours can be anything)

Format	In-class contact	+ Out-of-class	Total	Rough # modules
Intro to a topic	1.5 hr	—	1.5 hr	1
Workshop	3 hr	—	3 hr	1
Lecture series	5 × 1 hr	—	5 hr	~5
5-week series	5 × 2 hr	optional	~10 hr	~5
8-week course	8 × 3 hr	8 × 1 hr	~32 hr	~8
12-week course	12 × 4 hr	(in the 4)	~48 hr	~12

These are **starting points, not a menu.** The module count falls out of (total hours ÷ how long each coherent chunk needs) — a 3-hour workshop is likely one module; a 12-week course, about twelve.

The formula

Hours = Total quantity ÷ Average-per-hour

For each reading, video, demo, or practice activity: estimate how much a student gets through in an hour (**average per hour**), how much there is (**total**), and divide.

Benchmark rates (starting estimates — tune to your audience)

Readings *(from the Georgetown tool — pages/hour, slower for denser material):*

Material	Pages/hour	What the student is doing
Dense / technical (textbook-style)	~20	**Studying** — reading to learn and retain
Article (HBR-style)	~30	Reading closely, but not to memorize
Web / lighter reading	~35	**Reading** — following the argument once

*Name the purpose or the number means nothing: it's the single biggest factor here, far bigger than how fast your people read. Measured adult silent reading of non-fiction averages about 238 words per minute, but measured **study** rates run from roughly 170 down to 40 wpm depending on the task (Brysbaert 2019, a meta-analysis of 190 studies). At ~450 words a page, the rows above work out to about 150, 225 and 260 wpm — so "~20 pages/hour" is a studying rate and "~35" is a reading rate. They're in the right places; just be honest about which one you're asking for.*

Watch / listen *(count at real length):*

Activity	Rate
Recorded video / lecture	1× its length (a 12-min video = 0.2 hr) + note time
Synchronous session / live demo	1:1 (1 hr = 1 hr)
Podcast / audio	1× its length

Hands-on (the part academic tools miss — estimate per rep, then tune):

Activity	Starting estimate
Guided hands-on rep (a brew, a build, one exercise)	~15–20 min each
Practice / worksheet set	estimate by # of problems
Paired / group activity	real time it runs

Two of those rows can't be used at the §3.5 fit-check — "real time it runs" and a 1:1 live demo both need materials that don't exist until §6. That's expected: at §3.5 you're sizing, so estimate; at §6a step 6 you're budgeting, so measure.

Shared equipment changes the arithmetic. If groups share a machine, a bench or a station, elapsed time isn't the length of the task — it's **machine-minutes × number of groups ÷ number of machines**. Three pairs sharing one mill at twenty minutes each is a sixty-minute block, not a twenty-minute one. Plan a bench task for the groups who are waiting, or that time becomes dead air.

(The original academic rates — reflective response ~ 1 hr, peer responses ~ 2/hr — are kept here only for reference; most won't appear in a hands-on workshop.)

How long will this take *them*? Don't estimate — instrument.

The benchmarks above get you a starting number. They can't get you the right one, because the person estimating is the person who already knows how to do it.

You will underestimate, and your expertise is the reason. This is a measured effect, not a figure of speech: when experts, intermediates and novices were each asked to predict how long novices would take on a task, **the more expertise someone had, the worse their prediction was** — and standard debiasing instructions didn't fix it (Hinds, 1999, *Journal of Experimental Psychology: Applied*). People at an intermediate level were the most

accurate. Worse, experts also write instructions at a higher level of abstraction, so the same expertise that mis-times the task also *specifies* it in a way that slows novices down (Hinds, Patterson & Pfeffer, 2001, *Journal of Applied Psychology*).

Padding won't save you. When thesis students predicted their own submission dates, the average prediction was 34 days and the average actual was 56. Fewer than half finished even by their own **worst-case** date — and predictions made deliberately pessimistically were no more *accurate* than the plain ones (Buehler, Griffin & Ross, 1994, *Journal of Personality and Social Psychology*). Adding 50% to a number you made up gives you a bigger made-up number.

There is no multiplier to reach for. You'd want a rule like "novices take three times as long." It doesn't exist. The measured expert-novice gap ranges from *no difference at all* to roughly fourfold depending on the domain and the task, and the famous "four times" figure comes from a study comparing **one** expert with **one** novice. Any general multiplier would be invention dressed as evidence.

So do this instead:

1. **Run the activity yourself with a clock running.** Write down the wall-clock time — don't trust the memory of it afterwards; recalled durations are themselves systematic underestimates (Roy, Christenfeld & McKenzie, 2005, *Psychological Bulletin*). That number is your **floor**, not your estimate.
2. **Put it in front of two or three people who actually resemble your learners**, and log their time from a timestamp. Don't ask them afterwards — in one study learners over-reported how long work took them 78% of the time, by a median factor of about 1.45, while 79% of them believed they tracked their time accurately (Hart, Warren & Edwards, 2025, *PLOS ONE*).
3. **Keep the ratio between your time and theirs.** That is your correction factor — for your domain, your materials, your people. It is the only one with any standing, because it's built from comparable cases rather than from a general rule.
4. **Write it down and reuse it.** By your third or fourth activity you're estimating from your own data instead of your intuition, which is the only correction this problem actually responds to.

None of this is exotic: timing real learners is a standard step in systematic instructional design (the small-group stage of formative evaluation collects completion time) — it's just the step almost everyone skips. If you genuinely can't find two representative people, use the slowest plausible rate from the table above, state the result as a range rather than a number, call it provisional, and measure it the first time you run the course.

How to use it

1. **List every piece of work** in the module (readings, watch/listen, hands-on).
2. **Set an average-per-hour** for each from the benchmarks above (adjust for your people).
3. **Enter the total quantity.**
4. **Compute hours** (total ÷ average) and **sum.**
5. **Compare to the container** and adjust: overflow → split, cut depth (§7), or move to pre/post-class; too thin → merge or add practice.

The table

Type	Item	Average per hour	Total	Hours
Reading	…	pages/hr	pages	…
Watch/listen	…	1× length	minutes	…
Hands-on	…	reps/hr	reps	…
			Module total	**… hrs**

Worked example — Coffee "Setup & First Brew" (90-min workshop session)

Type	Item	Avg/hr	Total	Hours
Watch (pre)	Grind-size + freshness videos	1× length	~8 min	~0.15 (pre-class)
Live	Welcome + one model + taste reference	1:1	15 min	0.25
Live	Gear & beans walkthrough	1:1	15 min	0.25
Hands-on	Brew-along demo	1 brew/0.25 hr	1	0.25
Hands-on	Paired fill-in brew	1 brew/0.3 hr	1	0.30
Hands-on	Two-cup compare + tasting	real time	20 min	0.35
			Live total	**~1.4 hrs**

Read-out: ~1.4 hrs of live activity against a 1.5-hr workshop block — fits, but tight once you add transitions. If it had hit 2 hrs, move the two-cup compare to post-class rather than cut it. Pre-class video (~0.15 hr) sits well under any reasonable pre-work cap.

(Want the live spreadsheet behavior? Recreate these columns in your own spreadsheet tool.)

Pre-Class — Session 3 (Structure the Module)

~30 minutes.

Do this before Session 3

1. **Decompose your course on the worksheet, then build your Master Design Document from it** (Deliverable B). All seven steps first, for every course objective — as you saw it filled for coffee; then bring the module list across into the MDD, add topics and materials, name the activities, and check it against **Rubric 5.2** before you call it done.
2. **Choose the ONE module you'll build.** Just one — not the whole course.
3. **Have your Decisions doc in hand.**
4. **Read the guide, Section 5 (the three packages)** — including the accessibility (UDL) part.
5. **Unzip the two module folders beside your bundles.** `Coffee_Module1_filled.zip` is the coffee course's Module 1 — the three packages laid out, every file named, three built to depth (the reference brew, the paired fill-in card, one gold-tier segment). `Module_Skeleton_TEMPLATE.zip` is the blank version of the same thing; in class you will unzip it inside your course folder and make it yours. Glance at the coffee one before class.
6. **Make your course a folder.** Put your Decisions doc, your Objectives and your MDD together at the root of one folder — that is what an agent reads. Copy `AGENT_INSTRUCTIONS.md` (from `templates/`) in beside them and rename it for your tool:
   - rename it `AGENTS.md` — the universal name; VS Code Copilot, Codex and most agents read it
   - Claude Code → `CLAUDE.md`

   Then run the trace-back on it — **AI-4.1**, the way you saw in class: *"Read the Objectives doc and the MDD in this folder. Report each →CO# tag that points at no course objective, and each course objective no module serves. List only, with file paths. Change nothing."* **Bring what it flagged.** *(No agent tool? Paste the chat version from the Directing AI card — same test, same answer.)*

Come ready to

- Be taught what a module is made of — the three packages — on the coffee module, then start your own: the module folder and the one-page **Module Skeleton Map** in your fill-in file. (You produce the content *next* week.)

Quality Checklists — Quick Card

*The three "is this good enough yet?" checks, in the order you'll need them. Run each one **before** you move on — a gap caught here is minutes; the same gap caught in Week 5 is a rebuild. (These also live in Guide §10.)*

Every objective

Run this on each objective as you write it — course-level and module-level alike.

- ☐ **Measurable verb?** You can watch a student do it. Not *understand, know, learn, appreciate, be familiar with, grasp.*
- ☐ **Student-focused?** It says what the *student* will do, not what you will cover.
- ☐ **Right altitude?** The Bloom level matches what they actually need — higher isn't automatically better.
- ☐ **Laddered to a course objective?** Every module objective is tagged →CO#.
- ☐ **Assessable?** You can name, right now, the thing a student would produce to prove it.

  If an objective fails the first box, fix it before anything else. Every later artifact inherits the verb.

Your MDD

Run this once the whole blueprint is drafted, before you build anything.

- ☐ **Every module has objectives / topics / materials / activities?** No blank cells.
- ☐ **Objectives tagged →CO#?** Anything that traces to nothing is either scope creep to cut, or an objective you forgot to write.
- ☐ **Activities named, not vague?** "Students practice" is not buildable. *"Pair exercise: fill-in-the-blank, ~70% provided"* is.
- ☐ **Practice-before-assessment present?** Nobody is assessed on something they haven't rehearsed.
- ☐ **Values shown?** Mapped to specific activities, not restated as a list.

One module

Run this before you call a module done.

- ☐ **All three packages present?** Instructor materials, student materials, teaching guide.

- ☐ **Portable — could someone *else* teach it from the guide alone, cold?** This is the real bar, not "could I teach it."
- ☐ **Scaffolded, not blank?** Practice has fill-ins, TODOs or partial support — never an empty page.
- ☐ **Correct / verified?** You ran it, did it, or checked it yourself. "The AI wrote it" is not a verification.
- ☐ **Visual where it helps?** Paired with text, never replacing it.
- ☐ **A way for students to self-check?** They can tell whether they got it right without you.

Two things these checklists assume

You pass your own self-check first. Writing a module and being able to teach it are different capabilities. Take the self-check yourself — at your desk, where a gap costs ten minutes.

Teach what you assess, with no exceptions. Walk each checklist item back to the moment it's taught. Assessing something the course never taught feels like a trap to the student, and it's the one error they won't forgive.

Pre-Class — Session 4 (Produce With GenAI)

~30 minutes.

Do this before Session 4

1. **Finish your module folder and your Module Skeleton Map** — every file renamed to your form or deleted; the map's parts C and D done: the rung for each student piece, and the agenda sketch with first estimates. If it runs past your container, fix it on the map now. **Send the map in by the day named in class** — the maps are read before Session 4, and the session opens with what they showed.
2. **Have your Decisions doc in hand — and your course folder** — you'll feed them to GenAI in class.
3. **Read the guide, Section 6 (Produce With GenAI)** — including **6e, the loop with an agent (AI-6.1 to AI-6.7)** — skim **6b (the two councils)** and **6d (pick the tool that fits)**, and read the short **Section 7 (depth)**. Section 8 you met last week; this week it gets *built* (AI-8.1 to AI-8.4).

Come ready to

- Be taught the production loop in full and watch it run live; catch a planted error. **You do not produce in class** — the build is this week's work, at your desk, from the loop.
- Start your **AI-direction + verification log** (the template is in your fill-in file): one row per run, chat or agent.

Reviewing AI Output — Two Red Flags

A take-home card for checking anything an AI drafts for you. "Verify everything" is hard to act on; these two tells show you ***where to dig first.***

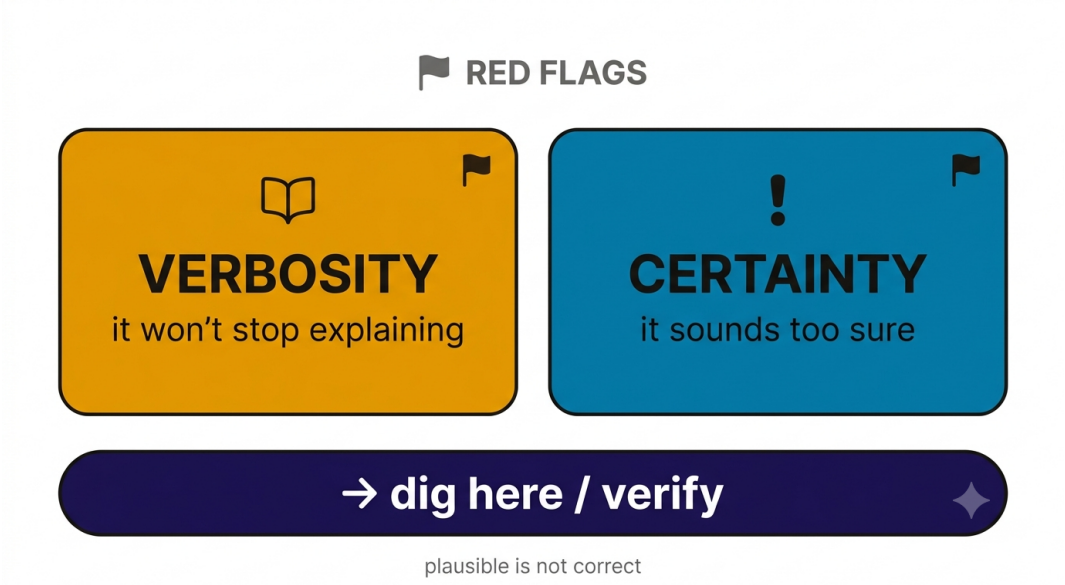


Figure 1: Two AI red flags — verbosity ("it won't stop explaining") and certainty ("it sounds too sure") — point to: dig here and verify. Plausible is not correct.

The two flags

1. Verbosity — it's over-explaining. Reasoning models especially pad, restate, and pile on caveats. Length is often hiding thin or wrong substance. **When it won't stop explaining, slow down and check what it's actually claiming.**

2. Certainty — it sounds *too* sure. Hedge-free, confident statements with no "it depends" are a tell, not a comfort. The most confident sentence is often the one to verify. **Overconfidence is where it's most likely confidently wrong.**

> **Rule of thumb:** *the more it over-explains and the more certain it sounds, the harder you check.* Plausible is not correct.

What to do when a flag trips

- Trace the specific claim back to a source you trust (the real tool's behavior, the data, the docs).
- Ask it to state its uncertainty: "What in this are you least sure about?"
- Cross-check with a second prompt or a second model.

Remember

The AI **advises or drafts; you decide and verify** (the Command Rule). It's wrong a meaningful fraction of the time, in *subtle* ways — these two flags are your fast triage for finding it.

Bounding the Answer Space — a worked 5b (Intro-to-ML)

Guide §6a, step 5b. This is what "bound the answer space" actually produces, end to end, for one divergent assignment. Step 5a covers assessments with one right answer — a quiz item, a coding exercise — where the key falls out of the complete solution. This file is for the other kind: ***many answers are valid, so there is no key to derive.***

---

1. The assignment (the divergent one)

Intro-to-ML's convergent work is easy to assess: the notebook runs or it doesn't. **CO5 is not like that.**

> **CO5** — Judge model fit using MSE/RMSE/R^2 and **communicate results to stakeholders.**
>
> **The assignment:** You've trained the Week 1 housing model. Write a **one-page model report** for a non-technical stakeholder who has to decide whether to use it. Say what it does, how well it works, and where it shouldn't be trusted.

There is no single correct one-page report. Two good students will write genuinely different ones. That's the definition of divergent work — and the reason step 2's "build the complete solution first" doesn't rescue you here.

---

2. The axes (write these first — they come from you, not the AI)

Before prompting anything, name the three or four things you'd comment on in **any** submission. These come from the annotated exemplar you built in step 2:

#	Axis	The question it answers
A1	**Leads with the decision**	Does the stakeholder know what to do by the end of the first paragraph?
A2	**Honest about error**	Is the model's error expressed in units the reader cares about (dollars), not just R^2?

#	Axis	The question it answers
A3	**Names the limits**	Does it say where the model shouldn't be trusted — and is that specific, not hedging?
A4	**No unexplained jargon**	Can a reader who doesn't know what "residual" means still follow it?

Why first: if you let the AI generate the range before you've named the axes, you end up grading against whatever the model happened to vary.

---

3. The prompt (run it eight to ten times, varying the approach)

> "You are a student in an introductory applied machine-learning course for working professionals. You have trained a linear regression model on the California Housing dataset (RMSE about $50k, R^2 about 0.6). Write the one-page model report described below, for a non-technical stakeholder deciding whether to use the model.
>
> `[paste the assignment]`
>
> Write it as **one plausible student would** — not as a model answer. Vary your approach each time I ask: this run, be the kind of student who [**leads with the methodology / is over-confident about accuracy / buries the recommendation / writes well but avoids numbers / over-hedges everything**]."

Run it separately per variant. Asking once for "ten different versions" collapses them into one voice with ten hats on — you get variation in wording, not in *approach*, and approach is what you're trying to chart.

---

4. The feedback envelope (what came back, sorted by axis)

Axis	Strong	Typical	Off-base
A1 — leads with the decision	Opens with "I'd recommend using this for initial estimates, not final pricing."	Recommendation appears, but in the last paragraph after the methodology.	No recommendation at all — describes the model and stops.
A2 — honest about error	"Typically within about $50k of the real price — so it's useful for ballparks, not for setting a listing price."	Reports RMSE and R^2 correctly but never converts them into stakes.	"The model is 60% accurate." (R^2 is not an accuracy percentage.)
A3 — names the limits	Names a specific boundary: the data is 1990s California; it won't transfer to today's market or another state.	Generic caution — "no model is perfect," "more data would help."	Claims the model is ready for production use.
A4 — jargon	Explains or avoids every technical term; a manager could read it cold.	One or two terms slip through unexplained (*residual*, *feature*).	Written for the instructor, not the stakeholder.

The most common failure was A1 — burying the recommendation under the methodology — which is worth knowing *before* you teach the session, because it tells you what to emphasize in the segment.

---

5. The feedback bank (written before anyone submits)

Two or three sentences per band, in the course's voice. Most of your marking is now done in advance:

A1 — leads with the decision

- *Strong:* "Your first sentence tells me what to do. That's exactly right — a busy reader may not get past it, and they don't need to."
- *Typical:* "Your recommendation is sound, but it's in the last paragraph. Move it to the top: the reader decides whether to keep reading in about ten seconds."
- *Off-base:* "You've described the model accurately but never said what you'd do with it. A stakeholder can't act on a description — what's your

recommendation?"

A2 — honest about error

- *Strong:* "Converting RMSE into dollars is the move that makes this useful. That's the sentence the reader will remember."
- *Typical:* "The numbers are right, but what do they mean for someone making a decision? Try: 'typically within about $X of the real price.' "
- *Off-base:* "R^2 isn't an accuracy percentage — it's the share of variance explained. Worth re-checking, because a stakeholder will take '60% accurate' literally."

A3 — names the limits

- *Strong:* "Naming the 1990s California data as the boundary is specific and honest. That's the difference between a caveat and a disclaimer."
- *Typical:* " 'No model is perfect' is true of everything, so it tells the reader nothing. What specifically would make *this* model wrong?"
- *Off-base:* "This reads as production-ready. Given the error range, what would have to be true before someone relied on it?"

A4 — jargon

- *Strong:* "A manager could read this cold. That's harder than it looks."
- *Typical:* "Two terms slipped through — *residual* and *feature*. One clause each and you're clear."

---

6. Where this lives, and what students see

- **This file** (the envelope and the bank) → `Instructor_Materials/reference_solution/solutions_and_key.m` It is for you, not for them.
- **The student-facing half** → a short reader's checklist in `Student_Materials/guided_practice/`, phrased as questions they run against their own draft: *Does my first paragraph say what to do? Is my error in dollars? Have I named where this shouldn't be trusted? Would my manager understand every word?*

Same four axes, two audiences. The checklist lets them self-correct before submitting, which is the point: **it's a feedback envelope, not a grading envelope** — it tells you what to say, not what to score.

---

7. What this method can't do

The limitations note in §6a applies, and it matters most here. The LLM is **not your students**: it writes like a competent median, so it will miss both the genuinely creative answer and the genuinely confused one. Treat the envelope

as a way to *widen* your expectations and pre-write your feedback — never as the definition of the acceptable set. If a student turns in something good that isn't in your table, the table was incomplete; that's normal.

Update it after the first run (§9) with what students actually submitted. The AI's version was a stand-in until you had real work to look at.

Pre-Class — Session 5 (Evaluate & Improve, and the Recorded Teach-Back)

~30 minutes. Nothing is due on the day of Session 5 — the build and the recording are due together two weeks after it.

Do this before Session 5

1. **Keep building** your one module with the production loop — complete version first, the practice derived from it. The floor is: a verified complete version · one practice · a pre-class study guide · one teaching-guide segment · a self-check. **The minimum that counts is one segment, complete** — its script and the complete version it narrates. Keep the AI-direction + verification log (template in your fill-in file).
2. **Read the guide, Section 9 (Evaluate & Improve)** and the closing section, **Teaching It Well**. Session 5 teaches §9 with your own recording as the first run.
3. **Read *Recording Your Teach-Back*** (one page, in your IN_CLASS packet) — what to record, how, and what to send.
4. **Bring your module folder as it stands, and your questions about the build.**

Come ready to

- Learn what you do after you teach a course for real — the four levels, the retro, the two AI jobs around it — and write your evaluate-and-improve plan (one of the rubric's must-rows).
- Hear the eight habits of *Teaching It Well* and pick two to try in your recording.

Recording Your Teach-Back

How the course ends. Two things are due together, two weeks after Session 5: your module, built to the floor, and a ten-minute teach-back of one segment of it, recorded. You get the feedback form back — the module rubric, filled in — the week after.

1. Build first

Build your one module to the **minimum-viable floor** with the production loop: a verified complete version · one practice derived from it · a pre-class study guide · one teaching-guide segment · a self-check. **If that is more than the time allows, the minimum that counts is one segment, complete** — its script in your teaching guide, and the complete version it narrates. Keep the AI-direction + verification log as you go (the template is in your fill-in file).

2. Before you record

- Run **AI-9.1** (the Directing AI card): *"Here is my Decisions doc and my module. You are one of the students it describes, and it didn't work for you. You're not hostile, just honest. What went wrong?"* Treat what comes back as things to watch for — never findings.
- Pick **two** of the eight *Teaching It Well* habits to try: show the whole picture first · small sure steps · an analogy or a story · draw it live · go slow where it matters · teach warm · ask, don't just tell · leave them wanting the next room.

3. Record ten minutes

- **One segment**, taught from **your own teaching guide**, exactly as you would teach it in your real course. Ten minutes; a little under is fine.
- Any recorder you have — Teams, Zoom, Loom, your phone. Camera optional. Share your screen if the segment shows something (a notebook, a diagram, a sheet).
- Teach to a student, not to the reviewer. Do not stop to explain your choices; the guide and the retro do that.

4. Watch it back once — this is your retro

With your guide open beside the video:

1. **Time it from the timestamp**, not from memory, and write down how it compared to the estimate on your agenda.
2. **Mark every moment you said something that is in your head and not on the page.** That gap is where a course dies when you leave the room; each one is a line to add to the guide.

3. **Write what you would change**, and where it lands — which MDD row or module file. Half a page is enough.

Then, if you like, **AI-9.2**: *"Here are my notes from the run. Turn them into specific changes, each naming the MDD row or module file it lands in. Don't propose anything my notes don't support."*

5. Send, by the date given in class

- the recording (a link or a file)
- the segment of your teaching guide you taught from
- your half-page retro
- your evaluate-and-improve plan (written in Session 5; it goes in your MDD as a short note)
- your AI-direction + verification log
- your module folder, as a zip

6. What comes back

The **feedback form** — Rubric 5.3 filled in: the artifact rows from your folder, the teach-back rows from the recording with a timestamp at each head-not-page moment, your retro and plan, whether Deliverable C is complete or the one sentence that would complete it, and one strength and one thing to sharpen, each tied to a rubric row. Met or not yet — nothing is scored.

Optional, for a later cohort or if you want it: swap recordings with one peer, watch theirs with their guide segment open, and send two lines — a strength and a sharpen, each naming a rubric row — plus one head-not-page timestamp.

Teaching It Well — Eight Principles

*The method builds a course anyone can teach; these are how you make it **unforgettable**. Apply them when you teach your module back — and in every course you deliver. (Full version: Guide, "Teaching It Well.")*

1. **Show the whole picture first.** Give the map before the details — every piece then has a place to land.
2. **Break it into small, sure steps.** Pieces small enough to feel doable; confidence compounds.
3. **Make it land — an analogy for clarity, a story for memory.** Both in your own voice, never borrowed.
4. **Draw it live; make it interactive.** Build the diagram in front of them and narrate as you go; favor interactive visuals over static ones.
5. **Go slow, and spend energy where the content earns it.** A deliberate pace; lift your energy for what matters.
6. **Teach warm — be the first to say "I didn't get this either."** Vulnerability makes not-knowing safe — the only place learning starts.
7. **Ask, don't just tell.** Questions that get at the heart of an idea; a question they answer sticks.
8. **Leave them wanting the next room.** Open loops; make the subject feel bigger than the session.

Rubrics — What "Done" Looks Like

*Each deliverable is judged against a **completion checklist** — met / not-yet, not a letter grade. **Not-completed means not-yet:** you get specific feedback and revise until it meets the bar, with no penalty. Rows marked **(must)** are required. Keep this beside you and self-check before you submit.*

Rubric 5.1 — Course + Module Objectives (Deliverable A)

Criterion	Meets standard
Measurable	Uses Bloom action verbs; no "understand/know/appreciate."
Student-focused	Describes what the *student* can do, not what the instructor provides.
Right altitude	Aimed at the highest level the content honestly supports (Apply/Analyze/Create for a skills course).
Laddered	Module objectives clearly roll up to course objectives. *(Course objectives come Week 1; module objectives Week 2, once you have modules.)*
Assessable	You could write a test item or task directly from each objective.

Rubric 5.2 — Master Design Document (Deliverable B)

Criterion	Meets standard
Complete	Every module has objectives, topics, readings, activities, assessments.
Mapped	Each module objective traces to a course objective.
Practice + feedback	Every module includes a way to practice *before* being assessed.
Values mapped	Each value from your Decisions doc is mapped to a **concrete feature** of the course — not restated. "Meet people where they are" becomes "hints remind of purpose, never hand over the answer."
Coherent sequence	Concepts build on each other; prerequisites are explicit.

Criterion	Meets standard
Practices audited *(must)*	The seven-practices audit was run; each gap found became a design task addressed in the MDD.

Rubric 5.3 — One Complete Module, Built and Taught (Deliverable C)

Judged in **two parts** — an **artifact review** of your module folder, plus the **live teach-back** in Week 5.

Artifact review — from your module folder:

Criterion	Meets standard
Three packages present *(must)*	Instructor materials, student materials, and a teaching guide all exist (to the minimum-viable floor).
Scaffolded *(must)*	Student practice uses appropriate fill-in/TODO support, not blank pages.
Correct *(must)*	Any code/worked examples run and were verified, not just plausible.
AI-direction + verification log *(must)*	A short log shows what you asked GenAI for, what it got wrong, and how you caught and fixed it.
Accessible *(must)*	Offers more than one way in across UDL's three principles — represent / engage / show mastery — not color-alone.
Evaluate-&-improve plan *(must)*	A short plan appraises the course against evaluation criteria and names improvements for the next run.
Visual	Uses visualization where it aids understanding.
Self-assessment	Includes a way for students to gauge their own understanding.
Answer space bounded *(if open-ended)*	For divergent assessments, the range of valid answers was charted ahead (by bounding the answer space, §6a step 5b) — used to *widen* the feedback envelope, not define it.

Teach-back — live, or recorded with instructor feedback:

Criterion	Meets standard
Portable (the key test) *(must)*	A different instructor could teach it from the guide alone — demonstrated by the teach-back, watched with the guide open; each head-not-page moment is a line to add.
Teach-back delivery *(must)*	The 10-minute segment was clear, on-time, followed your own teaching guide, and applied the Teaching It Well principles.

Self-Assessment — Can I Do These?

Check yourself as you go. If you can't tick it, that's where to focus.

After Week 1 — Decisions & Objectives

- ☐ I have a one-page Decisions doc — **values**, audience, objective, depth, scope, tone, tools, and my **container** (session length x sessions = contact hours).
- ☐ I've filled the three that went home with me: **scaffolding form**, **who teaches it**, and **gates** (write "none" if your course has none).
- ☐ My objectives use observable Bloom verbs (no "understand/know").
- ☐ Each module objective ladders up to a course objective.
- ☐ I could write a quick check or task straight from each objective.

After Week 2 — Master Design Document

- ☐ I ran the seven-practices audit and filled (or flagged) each one.
- ☐ I decomposed my course into right-sized modules that fit my time.
- ☐ My MDD has a course-header block + one table per module.
- ☐ My activities are **named and concrete**, not "students practice."
- ☐ I chose the ONE module I'll build.

After Week 3 — Structure

- ☐ My module folder has all three packages laid out (skeleton).
- ☐ I planned the I-do → we-do → you-do scaffolding for each piece.
- ☐ I built in access across all three UDL principles (represent, engage, show mastery).
- ☐ I chose a module small enough to build to the floor.

After Week 4 — Produce

- ☐ I built the complete solution first, then made practice by removing from it.
- ☐ I used GenAI with the guardrails and verified every line.
- ☐ I ran a Review Council review and revised.
- ☐ I kept an AI-direction + verification log (what I asked, what it got wrong, how I fixed it).

After Week 5 — Teach-Back

- ☐ I built my module to the minimum-viable floor.
- ☐ I taught a 10-minute segment from my own teaching guide.
- ☐ **Portability:** someone else could teach my module from my materials alone.

- ☐ Any code/worked examples are correct and were verified.
- ☐ My module has a way for students to check themselves.
- ☐ I have a short plan to evaluate the course and improve it next run.

The deliverables

- ☐ **A** — Objectives
- ☐ **B** — Master Design Document
- ☐ **C** — One complete module, built and taught

Glossary — The Course Master Method

The terms you'll hear across the five weeks.

- **Decisions document** — A one-page record of the choices only you can make (audience, objective, depth, scope, tone, tools). Written first; everything downstream inherits it.
- **Decisions before production** — The core rule: decide *what* and *how deep* before producing any material.
- **Learning objective** — A statement of what the *student* will be able to *do*, in an observable verb.
- **Bloom's Taxonomy** — Six levels of cognitive demand (Remember → Understand → Apply → Analyze → Evaluate → Create), each with measurable verbs. Used to set the altitude of an objective.
- **Course-level vs. module-level objectives** — Course objectives describe the end of the whole course; module objectives ladder up to them (tagged →CO#).
- **Seven research-based practices** — A design audit: Activate Prior Learning, Organize Knowledge, Motivate, Develop Mastery, Practice & Feedback, Student Development & Climate, Self-Directed Learners.
- **Decompose into modules** — Breaking a subject into right-sized, sequenced modules that fit your available time.
- **Container** — Your course's real schedule (e.g., a 3-hour workshop, a 12-week course), measured in **contact hours**.
- **Contact hours** — An estimate of how long student work takes; used to check a module fits its container (`Hours = Total ÷ Average-per-hour`).
- **Master Design Document (MDD)** — The single source of truth: a course-header block + one table per module (objectives, topics, materials, activities).
- **The three packages** — Every module ships **Instructor Materials**, **Student Materials**, and a **Teaching Guide**.
- **LLM** — A large language model: ChatGPT, Claude, Gemini and the like. When this course says "the AI," that's what it means.
- **System prompt** — The standing instruction you give an AI tool at the start of a project, so you don't retype your context every time. The two councils are written out as system prompts in Guide §10.
- **Altitude** — How high an objective aims on Bloom's ladder: *list* is low, *design* is high. Neither is better; they're for different goals.
- **Gates** — Anything a student must pass before they're allowed to do the next thing. Most courses have none. For safety-critical skills, decide this first — and note **a gate is completion, not a grade.**
- **Scaffolding form** — What your three rungs actually are in your medium: you demonstrate it, then you do one together, then they do one alone.
- **Gradual-release scaffolding** — *I do → we do → you do.* Support decreases across the course (e.g., complete example → fill-in-the-blank → independent).

- **Hints** — Guidance you leave for students when you're not in the room — a reminder of purpose, never the answer.
- **Minute-by-minute agenda** — A timed Time/Activity/Details plan for a live session, in the teaching guide.
- **Teaching-guide depth (floor vs. gold)** — *Floor:* agenda + talking points. *Gold:* verbatim, fully-scripted segments. Pick the level that makes the course teachable by whoever will teach it (the **portability** test).
- **Depth rule** — Go only as deep as your audience's goal needs; stop where deeper wouldn't help them. Heuristic: keep asking "why" until you'd be comfortable saying "I don't know" to the class.
- **GenAI guardrails** — You decide and verify; AI drafts. It's confidently wrong often enough that you check everything.
- **Feedback envelope** — For work with many valid answers: your axes (the three or four things you'd comment on in any submission) against three bands — strong, typical, off-base — with a real example in each cell. Built *before* anyone submits, so most of your feedback is written in advance. It tells you what to **say**, not what to score.
- **Beat** — One timed sub-block of a segment script. "One slide per beat" means the deck tracks what you actually say, in order.
- **Review Council** — A multi-persona review (SME, student, instructional designer, professor) that scores a *finished* module on consistency, delivery, factuality, and teaching principles before it ships. (Distinct from the competing-interests *design* council you can use while planning.)
- **Threading** — How modules connect: a short intro that links back and a conclusion that pivots forward.
- **Portability** — The real test of a course: could someone else teach it from your materials alone?